

stats

EECS 280 Project 1: Statistics

Due 8pm ET Mon Jan 27, 2025. This is an individual project.

Winter 2025 release.

i IMPORTANT If you are retaking the course, please note the project has changed significantly this term. While we always suggest students retaking the course redo the entire project from scratch (as it is helpful practice), we nevertheless want to emphasize you must work from a fresh set of starter files.

Introduction

Statistical analysis is a key tool in many fields, from the natural and medical sciences to social sciences and business. It's used to summarize data, make inferences about populations, and test hypotheses. The intersection of statistical analysis and computation also underlies many emerging fields like machine learning and data science.

In this project, you'll implement a program that computes descriptive statistics and performs two-sample analysis on a given data set.

For example, consider data from [How Couples Meet and Stay Together \(HCMST\)](#), a study of how Americans meet their spouses and romantic partners. We can investigate the age (`ppage`) of survey respondents depending on whether or not they met their partner online (`q24_met_online`).

As shown below, our program displays descriptive statistics for respondents' age in the two groups. It also runs a 95% confidence interval procedure for the difference in means between the groups, meaning there is a 95% chance the interval computed from the sample contains the true value of the difference in means among the overall population. Overall, we can be reasonably confident the mean age of people who met their partner online is several years younger than those who did not.

```
$ ./two_sample.exe HCMST_ver_3.04.tsv q24_met_online 1 0 ppage
reading column q24_met_online from HCMST_ver_3.04.tsv
reading column ppage from HCMST_ver_3.04.tsv
Group A: ppage | q24_met_online = 1
```

```
count = 270
sum = 10687
mean = 39.5815
stdev = 12.2103
median = 40.5
min = 19
max = 86
  0th percentile = 19
 25th percentile = 29
 50th percentile = 40.5
 75th percentile = 49
100th percentile = 86

Group B: ppage | q24_met_online = 0
count = 2664
sum = 125721
mean = 47.1926
stdev = 16.1446
median = 46
min = 19
max = 95
  0th percentile = 19
 25th percentile = 35
 50th percentile = 46
 75th percentile = 58
100th percentile = 95

Confidence interval for mean(ppage | A) - mean(ppage | B):
95% [-9.31595, -6.05068]
```

Learning Goals

The learning goals of this project include C++ fundamentals, control-flow structures, vectors, procedural abstraction, and unit testing. It's also a chance to get used to the C++ toolchain and review your 100-level programming skills. No prior knowledge of probability or statistics is required or expected for this project.

Setup

Set up your project in your visual debugger. We recommend VS Code because it's easier to use. Many people use Visual Studio (Windows) or XCode (macOS).

During setup, name your project `stats`. Use this starter files link:

<https://eecs280staff.github.io/stats/starter-files.tar.gz>

[VS Code Tutorial \(recommended\)](#)[Visual Studio Tutorial](#)[Xcode Tutorial](#)

If you created a `main.cpp` while following the setup tutorial, delete it. You'll use the provided starter code for `two_sample.cpp` instead. You should end up with a folder with starter files that looks like this. (You may have already renamed files like `two_sample.cpp.starter` to `two_sample.cpp`.)

```
$ ls
Makefile          library.cpp  stats_public_tests.cpp
cats.csv          library.hpp  stats_tests.cpp.starter
cats.out.correct  stats.hpp   two_sample.cpp.starter
```

Here's a short description of each file in this project.

File	Description
<code>Makefile</code>	Helper commands for building and testing
<code>stats.hpp</code>	Function prototypes for statistics library
<code>stats.cpp</code>	Function implementations for statistics library
<code>stats_public_tests.cpp</code>	Public tests for the statistics library
<code>stats_tests.cpp</code>	Your tests for the statistics library
<code>library.cpp</code>	Provided code implementations
<code>library.hpp</code>	Provided code function prototypes
<code>two_sample.cpp</code>	Two-sample analysis program
<code>cats.csv</code>	A small dataset for testing the analysis program
<code>cats.out.correct</code>	Correct outputs of analysis program on cats.csv



Pitfall: Make sure you have set up your visual debugger before continuing.

[VS Code Tutorial \(recommended\)](#)[Visual Studio Tutorial](#)[Xcode Tutorial](#)

Statistics Background

This section provides an introduction to the statistical analyses you'll implement in this project.

Descriptive Statistics

Descriptive statistics are computed properties that quantitatively summarize the underlying distribution of measured data. For example, the *mean* of a data set is a measure of its central tendency whereas the *standard deviation* measures how spread out the data are. Details of several descriptive statistics are given in the [Statistics Functions](#) section below.

Sampling

We often estimate qualities of a large *population* by measurement of a relatively small *sample*. For instance, we might want to know how many US residents are currently in a relationship, the distribution of age among these people, and whether they met their partner online or in-person. Rather than ask over 300 million people these questions, we might survey a subset of a few thousand people.

There are many essential considerations for proper survey procedure. For example, obtaining a random, unbiased sample of survey participants is important to ensure the sample is representative of the overall population. On the other hand, special attention (e.g. oversampling) may be needed to ensure underrepresented groups are not marginalized. Survey designers should mitigate potential threats to validity such as response bias (e.g. survey answers may be affected by perceived social desirability of answers). These considerations are important, but beyond the scope of this project.

Two-Sample Analysis

We may also want to compare two different samples via statistical analysis and make inferences about the underlying populations they represent. We might survey different populations entirely separately, or partition a data set into subgroups based on specific criteria. For example, we might filter responses to the HCMST survey based on whether people met their partner online or not, yielding two samples representing distinct populations. Then, we could compute the mean age for each sample, as well as the difference between those means, finding that to be approximately 7.6 years older for the sample of people who did not meet their partner online.

However, inferences drawn from samples must be qualified in a way that accounts for the probability the observed difference is due to random chance (i.e. the uncertainty introduced by sampling). It would not be responsible to simply report the 7.6 point estimate as the difference in means among the underlying populations.

Bootstrap Confidence Intervals

Instead of reporting a single estimate based on a sample, we instead set up a procedure for computing a range of possible values (i.e. lower and upper bounds) defining a *confidence interval* at a particular *confidence level* of X%. The interpretation is as follows: *If we collected many samples (i.e. ran the survey many times), X% of the time the computed confidence interval would contain the true underlying value among the population.*

Of course, it's not practical to actually conduct the survey many times, so we instead use an approach called *bootstrap resampling*. The essential idea is that we simulate taking several samples by instead re-sampling (with replacement) from one original sample.

For example, assume the original sample contained individuals [A, B, C, D]. Bootstrap resamples of this could be [A, C, C, D] or [A, A, A, B]. Each approximates a plausible sample we could have drawn from the underlying population (i.e. a resample of [A, A, A, B] approximates a sample that happened to have more people like A and less like C or D).

We can compute the statistic of interest (e.g. the difference in means) for many resamples to obtain an approximate *sampling distribution* from which we can compute a confidence interval.

Let's apply this to our running example. We start with the original two samples of people who did or did not meet their partner online. Then, we repeat this process 1000 times:

1. Generate a bootstrap resample for each of the two original samples.
2. Compute the difference in means between the two resamples.
3. Record the computed value.

Finally, we determine the range containing the middle 95% of the values computed via resampling and report this as the 95% confidence interval.

The starter code for `two_sample.cpp` provides much of the overall structure for computing bootstrap confidence intervals and includes comments to guide you through implementing the rest.

You might also find this [video](#) on bootstrapping helpful, which covers the same basic ideas described above. While there are many approaches to find confidence intervals, we chose bootstrapping for this project due to its uniquely computational approach that is well-suited for a programming project.

Statistics Library

The `stats.hpp` file specifies a general-purpose statistics library, including declarations and RME interface specifications of several functions for computing descriptive statistics and filtering data. You'll write implementations for each of these functions in `stats.cpp`.

The data sets are stored as a `std::vector<double>`. If you haven't used vectors in C++ before, check out our [Vector Reference](#). C++ vectors are similar to Java ArrayLists, Javascript arrays, and Python lists.

Setup

Rename these files ([VS Code \(macOS\)](#), [VS Code \(Windows\)](#), [Visual Studio](#), [Xcode](#), [CLI](#)):

- `stats_tests.cpp.starter` -> `stats_tests.cpp`

Add a new file `stats.cpp` containing a "function stub" for each prototype in `stats.hpp`, as shown below. Adding stubs ensures the program can compile, even though not everything is finished yet. The `assert(false);` in each is a placeholder that triggers an error if an unfinished function were to run - you should remove them once you implement each function.

stats.cpp

```
1  // stats.cpp
2  #include "stats.hpp"
3  #include <cassert>
4  #include <vector>
5  #include <algorithm> // sort
6  #include <cmath> // sqrt, modf
7
8  using namespace std;
9
10 int count(vector<double> v) {
11     assert(false);
12 }
13
14 double sum(vector<double> v) {
15     assert(false);
16 }
17
18 double mean(vector<double> v) {
19     assert(false);
20 }
21
22 double median(vector<double> v) {
23     assert(false);
24 }
25
26 double min(vector<double> v) {
27     assert(false);
```

```
28 }
29
30 double max(vector<double> v) {
31     assert(false);
32 }
33
34 double stdev(vector<double> v) {
35     assert(false);
36 }
37
38 double percentile(vector<double> v, double p) {
39     assert(false);
40 }
41
42 vector<double> filter(vector<double> v,
43                     vector<double> criteria,
44                     double target) {
45     assert(false);
46 }
```

Once you've added the stubs, you can compile and run the public tests.

```
1 $ make stats_public_tests.exe
2 $ ./stats_public_tests.exe
```

The tests should compile successfully but won't pass until you've implemented each function.

⚠ Pitfall: Do not include a `main()` function in your `stats.cpp` file. Remember, `stats.cpp` is a library of functions that another module like `two_sample.cpp` will use.

Statistics Functions

This section provides background for each statistical method and tips for implementation.

General Tips

📌 Pro-tip: Your implementation of some functions may require sorting the input vector. You can use the `std::sort()` function from the `<algorithm>` library for this. Here's an example:

```
1 vector<double> v; // assume v contains some data
2 std::sort(v.begin(), v.end());
```

⚠ Pitfall: If you're getting errors like this, check out the [Comparisons tutorial](#).

error: comparison between signed and unsigned integer expressions

count(), sum(), mean()

The `count()` function returns the number of values in a data set.

The `sum()` function computes the *sum* of the values in a data set.

The `mean()` function computes the *arithmetic mean* of a data set:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

Here, $\bar{x}$ denotes the mean, x_i denotes the value of the i th data point, and n is the sample size.

💡 Pro-tip: Call `count()` and `sum()` as helper functions in your implementation of `mean()`.

median()

Computes the *median*, which is the “middle” value in a data set. In other words, it is the value for which half the data points lie below the value, and half lie above it. If the number of data values is odd, this is just the middle value when the data set is sorted. If the number of data values is even, then it is the average of the two values in the middle.

min(), max()

These functions return the *minimum* and *maximum* values in the data set.

stdev()

The `stdev()` function computes the *standard deviation*, which is a measure of the “spread” among data values. For instance, while the data sets `[1, 2, 3]` and `[0, 2, 4]` have the same mean of 2, the latter set has a higher standard deviation than the former one since the values are more spread apart. We specifically use the [corrected sample standard deviation](#), which is defined as:

⚠ Pitfall: Make sure that your code is doing floating-point rather than integer division.

```
1 double x = 1 / 4;    // integer division: x is 0
2 double y = 1.0 / 4;  // floating-point division: y is 0.25
```

When writing `stdev()`, use the `sqrt()` function, which calculates a square root.

```
1 #include <cmath>
2 // ...
3 cout << "the square root of 4 is " << sqrt(4) << "\n";
```

percentile()

A call to `percentile(data, p)` computes the *percentile* for a fractional value p , which is the threshold below which a portion of the data occurs. For example, `percentile(data, 0.3)` returns the value below which 30% of the data occur.

We use the percentile formula that is implemented in many software packages, which estimates the percentile for a fraction as follows:

1. Compute the *rank*, which is the approximate position in the (sorted) data set corresponding to the fraction:
2. Split the rank into its integer and decimal components i and f respectively, so that
3. Then the percentile is

where x_i is the i th data point from the sorted data set. Essentially, x is the weighted average of the i th and $i+1$ th data points in the sorted set, with the weight of each value determined

by the decimal component : a weight of for the th value and a weight of for the th value.

Pro-tip: Use the `modf()` function to break a double into its integral and fractional parts.

```
1  #include <cmath>
2  // ...
3  double pi = 3.14159265;
4  double intpart = 0;
5  double fractpart = 0;
6  fractpart = modf(pi, &intpart);
7  // intpart is now 3, fractpart is now 0.14159265
```

Pitfall: The formula here and the worked example below use indices that start with 1. You will need to adapt them for `vector` indexing, which starts at 0.

Pitfall: Consider an example dataset with 5 elements. For $p=1$, the above formula gives:

This includes a nonexistent datapoint - ! Mathematically, everything works out because it gets multiplied by 0. However, you must ensure your implementation code never even attempts to access out-of-bounds data, which can cause undefined behavior, including a program crash (regardless of whether it would be canceled out later in the formula).

Worked Example of Percentile Computation

Suppose that our data set consists of the values . What is the 40th percentile from this data set?

First, we sort the values to obtain the ordered sequence

Then, we calculate the rank for :

Splitting this into integer and decimal components, we have

We then calculate the percentile as

Thus, the 40th percentile is 29.

`filter()`

The `filter()` function separates out the portion of a data set where some measured criterion has a particular value. For example:

```
1 // Data set of temperatures measured at different locations.
2 // e.g. the first measurement was at location 0 with temperature 15.5
3 vector<double> locations = {0, 1, 0, 1, 1, 2, 2, 0, 1};
4 vector<double> temps = {15.5, 23.1, 7.8, 19.2, 22.6, 4.6, 1.9, 14.3, 18.0};
5
6 // Filter to the temperature measurements at location 1
7 vector<double> temps_1 = filter(temps, locations, 1);
8 // temps_1 is {23.1, 19.2, 22.6, 18.0}
```

Testing

There are two sets of unit tests for the stats module.

1. Public Tests in `stats_public_tests.cpp`

A minimal set of public tests that we provide (and that match the ones on the autograder). Passing these is a good first target, but is generally not sufficient to ensure correctness. Compile and run the public stats tests with:

```
1 $ make stats_public_tests.exe
2 $ ./stats_public_tests.exe
```

2. Your Tests in `stats_tests.cpp`

Write your own comprehensive set of unit tests in `stats_tests.cpp` for each of the statistics functions declared in `stats.hpp`. Use `assert()` to verify the results of the function are as expected when called on a variety of inputs, including any relevant special cases. Keep your tests organized by splitting them into separate helper functions, as in the provided example in the `stats_tests.cpp` starter file.

Compile and run your tests with:

```
1 $ make stats_tests.exe
```

You will submit `stats_tests.cpp` to the autograder. You must write at least one test for each function in `stats.hpp`, but it's a good idea to write more. Generally speaking, your tests should not produce output to `cout` - the autograder will ignore such output, and you should use `assert()` to verify expected behavior.

 **Pitfall:** Due to rounding errors, two floating point numbers we expect to be equal may be slightly different. This might happen while testing `stdev()`. Check out the [Floating-point Comparisons tutorial](#).

Debugging

Use your IDE's visual debugger for either the public tests or your own tests.

For example, you might set a breakpoint in the unit test that is failing, before your function from `stats.cpp` is called. Then you can use the debugger to step into the function and see what's going wrong.

Here's a reminder of how to configure the debugger to launch either the public tests or your own tests:

	Public tests	Your own tests
VS Code (macOS)	First, compile from the terminal using <code>make stats_public_tests.exe</code>. Set program name to: <code>\${workspaceFolder}/stats_public_tests.exe</code> Click the run button in the debugging tab of the left side panel.	First, compile from the terminal using <code>make stats_tests.exe</code>. Set program name to: <code>\${workspaceFolder}/stats_t</code> Click the run button in the debug of the left side panel.
VS Code (Windows)	First, compile from the terminal using <code>make stats_public_tests.exe</code>. Set program name to: <code>\${workspaceFolder}/stats_public_tests.exe</code> Click the run button in the debugging tab of the left side panel.	First, compile from the terminal using <code>make stats_tests.exe</code>. Set program name to: <code>\${workspaceFolder}/stats_t</code> Click the run button in the debug of the left side panel.

	Public tests	Your own tests
XCode	Include compile sources: <pre>stats_public_tests.cpp , stats.cpp , library.cpp</pre> Click the run button.	Include compile sources: <pre>stats_tests.cpp , stats.cpp library.cpp</pre> Click the run button.
Visual Studio	Exclude files from the build: • Include <code>stats_public_tests.cpp</code> • Exclude <code>stats_tests.cpp</code> , <code>two_sample.cpp</code> , <code>main.cpp</code> (if present) Click the run button.	Exclude files from the build: • Include <code>stats_tests.cpp</code> • Exclude <code>stats_public_tests.cpp</code> , <code>two_sample.cpp</code> , <code>main.cpp</code> (if present) Click the run button.

Submit

Submit `stats.cpp` and `stats_tests.cpp` to the Autograder using the direct link in the [Submission and Grading section](#).

Analysis Program

Our statistical analysis program is run like this:

```
$ ./two_sample.exe HCMST_ver_3.04.tsv q24_met_online 1 0 ppage
```

It takes command-line arguments for the data file name (`HCMST_ver_3.04.tsv`), a filter column name (`q24_met_online`), filter A/B values (`0` and `1`), and a data column name to analyze (`ppage`).

When run, it reads data from the specified file and columns and splits into groups on the filter column and A/B values. Then, it prints descriptive statistics for the data column on each group, as well as a 95% confidence interval for the difference in means between the two groups.

The program may also be run for testing purposes with no command-line arguments provided:

```
$ ./two_sample.exe
```

In this case, the program uses default arguments of `cats.csv`, `food`, `1`, `2`, and `weight`, performing an analysis of the weights of cats in the provided `cats.csv` sample data set according to which brand of cat food (i.e. #1 or #2) they eat.

Setup

Rename these files ([VS Code \(macOS\)](#), [VS Code \(Windows\)](#), [Visual Studio](#), [Xcode](#), [CLI](#)):

- `two_sample.cpp.starter` -> `two_sample.cpp`

If you created a `main.cpp` while following the setup tutorial, you should delete it.

Data Sets

The analysis program can be run with any input data file in a standard delimited format (e.g. a `.csv` comma-separated value file or `.tsv` tab-separated value file) with a header row containing column names.

Sample Data Set: Cats

We've provided a small data set in `cats.csv` for testing purposes:

```
1  name, food, weight
2  Fluffy, 1, 9.6
3  Socks, 1, 11.5
4  Mittens, 1, 9.9
5  Felix, 1, 10.1
6  Luna, 1, 11.2
7  Simba, 2, 9.2
8  Nala, 2, 7.8
9  Oliver, 2, 12.3
10 Bella, 2, 10.1
11 Milo, 2, 10.9
```

When the program is run without command-line arguments, it defaults to use this data set, analyzing the weights of group A (fed food #1) and group B (fed food #2). The correct program output for this case is provided in `cats.out.correct`.

Real Data Set: HCMST

[How Couples Meet and Stay Together \(HCMST\)](#) is a study of how Americans meet their spouses and romantic partners.

Want to try it out with real data from the [How Couples Meet and Stay Together](#) study?

1. Use the following `wget` link to download the data in tsv format:
`https://eecs280staff.github.io/stats/data/HCMST_ver_3.04.tsv` .
2. The variables in the study are the first line of the tsv file.
3. Another file called the codebook describes the variables. It can be accessed here:
https://stacks.stanford.edu/file/druid:ns183dp7831/HCMST_codebook_3_04.pdf.

Overview

Complete the code in `two_sample.cpp` to implement the two-sample analysis program. The starter code handles several tasks for you, including reading command-line arguments, reading data from an input file, and the high-level structure of the two-sample analysis. You should spend some time reading through the file to familiarize yourself with the provided code.

You will need to implement the following functions in `two_sample.cpp` :

- `print_descriptive_stats()` : Prints descriptive statistics for a given data set.
- `mean_diff_sampling_distribution()` : Returns an approximation of the sampling distribution of the difference in means between two groups, computed using bootstrap resampling.
- `confidence_interval()` : Returns a confidence interval for a given data set and interval width.

Note the provided file contains `// TODO` comments and `assert(false);` placeholders for each function you need to implement. You should remove these as you complete each function. We have also provided `// HINT` comments with specific guidance.

Program Walkthrough

Let's walk through complete example of the two-sample analysis program. First, we'll compile and run the program at the command line.

```
1 $ make two_sample.exe
2 $ ./two_sample.exe
```

Next, let's walk through the program and its output, step-by-step.

i Pro-tip: Note the spacing and formatting of the output in the example below. Your program should match this format exactly.

The program starts in `main()` . Since no command-line arguments are provided, the program defaults to using `cats.csv` for the filename, `food` for the filter column, `1 / 2` for the A/B criteria, and `weight` for the data column. These are passed into the `two_sample_analysis()` function.

The `two_sample_analysis()` function calls `extract_columns()` to read from the input file, which prints an informational message.

```
reading column food from cats.csv  
reading column weight from cats.csv
```

It then filters into groups A and B. It prints a header for group A, calls your `print_descriptive_stats()` function on group A, and prints an extra blank line. Then it does the same for group B.

```
Group A: weight | food = 1  
count = 5  
sum = 52.3  
mean = 10.46  
stdev = 0.838451  
median = 10.1  
min = 9.6  
max = 11.5  
  0th percentile = 9.6  
 25th percentile = 9.9  
 50th percentile = 10.1  
 75th percentile = 11.2  
100th percentile = 11.5  
  
Group B: weight | food = 2  
count = 5  
sum = 50.3  
mean = 10.06  
stdev = 1.70088  
median = 10.1  
min = 7.8  
max = 12.3  
  0th percentile = 7.8  
 25th percentile = 9.2  
 50th percentile = 10.1  
 75th percentile = 10.9  
100th percentile = 12.3
```

It then computes a 95% confidence interval for the difference in means between groups A and B by calling your `mean_diff_sampling_distribution()` and `confidence_interval()` functions. Then it prints a header and the confidence interval.

```
Confidence interval for mean(weight | A) - mean(weight | B):
```

95% [-0.28, 1.2005]

Testing

Ensure you've saved all files and compiled your most recent code.

```
$ make two_sample.exe
```

Run the program (which defaults to using the `cats.csv` sample data set) and save the output to a file with [output redirection](#).

```
$ ./two_sample.exe > cats.out
```

Compare saved output (`cats.out`) with the instructor-provided correct output (`cats.out.correct`). If the `diff` command finishes with no output, that means the files are identical.

```
$ diff cats.out cats.out.correct
```

You may also open `cats.out` in your editor for manual inspection or use the `cat` command to print it at the terminal. (Note that it's simply a coincidence the command is also called `cat` .)

You can also use the `Makefile` to [run all tests](#).

```
$ make test
```

Debugging

Use your IDE's visual debugger to track down issues in your implementation of `two_sample.cpp` .

For example, if your printed descriptive statistics don't match, you might set a breakpoint shortly before the first mismatched line. Or, if the confidence interval is computed incorrectly, you can strategically set a breakpoint in-between various steps of the algorithm to check at which point something goes wrong.

Here's a reminder of how to configure the debugger to launch the two-sample analysis program:

VS Code (macOS)

First, compile from the terminal using `make two_sample.exe` .

Set [program name](#) to:

```
${workspaceFolder}/two_sample.exe
```

Click the run button in the debugging tab of the left side panel.

VS Code (Windows)	First, compile from the terminal using <code>make two_sample.exe</code>. Set program name to: <code>\${workspaceFolder}/two_sample.exe</code> Click the run button in the debugging tab of the left side panel.
XCode	Include compile sources: <code>two_sample.cpp</code> , <code>stats.cpp</code> , <code>library.cpp</code> Click the run button.
Visual Studio	Exclude files from the build: • Include <code>two_sample.cpp</code> • Exclude <code>stats_public_tests.cpp</code> , <code>stats_tests.cpp</code> , <code>main.cpp</code> (if present) Click the run button.

If you choose to run the program with custom command-line arguments, you'll need to configure them in your visual debugger ([VS Code \(macOS\)](#), [VS Code \(Windows\)](#), [Xcode](#), [Visual Studio](#)). For example, for the HCMST example in the project introduction, you would use the following arguments: `HCMST_ver_3.04.tsv q24_met_online 1 0 ppage`.

Submission and Grading

Submit `stats.cpp` , `stats_tests.cpp` , and `two_sample.cpp` to the autograder using this direct autograder link: <https://autograder.io/web/project/3033>.

We will grade your code on functional correctness and the presence of test cases.

Testing

Run all the public tests locally, including the public stats tests, your stats unit tests, and the two-sample analysis on the cats data set.

```
$ make test
```

Requirements and Restrictions

Use only the provided `bootstrap_resample()` function declared in `library.hpp` to perform resampling with replacement and ensure you pass in the current 0-indexed iteration number - this is necessary to ensure consistent pseudorandom number generation for autograding purposes.

Acknowledgments

The original project was written by Andrew DeOrio, Spring 2015. It was revised in Fall 2024 by James Juett and Amir Kamil.

This project is based on research work by Rosenfeld, Michael J., Reuben J. Thomas, and Maja Falcon. 2015. How Couples Meet and Stay Together, Waves 1, 2, and 3: Public version 3.04, plus wave 4 supplement version 1.02 and wave 5 supplement version 1.0 [Computer files]. Stanford, CA: Stanford University Libraries.

This document is licensed under a [Creative Commons Attribution-NonCommercial 4.0 License](#). You're free to copy and share this document, but not to sell it. You may not share source code provided with this document.